

一. 程序设计语言

语言有哪些类型？python 和 java 分别都是什么类型？

命令式 (Imperative)

过程式 (Procedure)

面向对象 (object-Oriented)

声明式 (Declarative)

函数式 (Functional)

基于逻辑 (Logic)

程序设计语言一般可以分为以下几类：

- 机器语言：直接由机器识别和执行的二进制指令，一般不需要编译器。
- 汇编语言：使用助记符号代替二进制指令，需要转换为机器语言后执行。
- 高级语言（编译型语言和解释型语言）：使用类似自然语言的语法和高级抽象结构，需要编译或解释器将其转换为机器语言后执行。

Python 属于解释型语言，Java 属于编译型语言。因此，Python 属于高级语言中的解释型语言，Java 属于高级语言中的编译型语言。Python 属于函数式编程语言。

Java 通常被视为编译型语言，因为 Java 程序是通过编译器将 Java 源代码编译为字节码，然后由 JVM (Java 虚拟机) 解释执行字节码。但是，由于 JIT (Just-In-Time) 编译器的引入，有些人也将 Java 视为解释型语言或混合型语言，因为 JIT 编译器可以在执行过程中动态地将字节码编译为本机代码，从而提高程序执行效率。但总的来说，Java 还是以编译型语言的身份更为广泛地被认知和使用。

二. 计算理论

1. 规约与停机问题

– 不可判定问题

停机问题 (Halting problem)

给定任意程序 P，和为其输入的 X，

判定其输出为 true，如果 P 能够在 x 的输入下停止。

判定输出为 false，如果 P 在 X 的输入下无法停止。

可判定性 (Decidability)：存在一个算法，来确定每一个形式化的命题是真命题还是假命题。

2. 图灵机

– 图灵机编码

图灵可计算

如果一个自然数上的函数是可计算的，当且仅当存在一个计算它的图灵机。

即对于任意该函数定义域上的输入，都可以使得图灵机停机，并输出该函数要计算的值。

3.lambda 演算

– 什么是β规约？

基本规则：

即用 N 来替换参数 x

改规则可以一直持续应用于任何λ子项。

$(\lambda x . M)N \rightarrow M[N/x]$

重命名，替换

– 什么是β-redex？

β-可约项（β-redex）以 $(\lambda x . M)N$ 形式出现的λ项。

什么是 lambda 演算的“停机问题”？

判断一个 lambda 表达式有没有β-范式。

三. python

1.基础语言特性（包括容器）

1) 变量与基础数据类型

2) 函数：curry 化，高阶函数

3) 容器：

列表（List）：任意数据类型的值的序列（sequence）

序列迭代，序列解包，range，缺省参数，序列切片（Slicing），序列聚合（sum,min,max,any,all）。

字符串（String）：字符串是一种典型的数据抽象，由字符的序列构成，可以代表复杂的事物。

和 list 序列一样，有迭代、重复、索引、合并、查询数量等操作（语言的正交性）。

in 比较特殊，其用于查找是否是某个字串的子串。

转化字符串函数：str()

字典（Dictionary）：字典是键值对（Key-Value pairs）的集合。

字典访问：通过键访问值。

一个字典里的所有的键都必须是不同的。

一个字典里的键不能是一个列表或者字典（或者其他可变 mutable 的类型）。

值的类型不受限制。

字典的迭代

字典推导式：

{<key exp>: <value exp> for <name> in <iter exp> if <filter exp>}

Short version: {<key exp>: <value exp> for <name> in <iter exp>}

例子:index:

def index(keys, values, match):

"""Return a dictionary from keys k to a list of values v for which
match(k, v) is a true value.

```
>>> index([7, 9, 11], range(30, 50), lambda k, v: v % k == 0)
{7: [35, 42, 49], 9: [36, 45], 11: [33, 44]}
"""
return {k: [v for v in values if match(k, v)] for k in keys}
```

嵌套数据

树:

类型的闭包属性 (closure property)

一种构成复合数据的方式具有闭包属性当且仅当:

如果其生成的复合数据可以作为元素之一被用同样的方式进一步复合。

比如, 列表可以作为元素被另一个列表所包含。

树是一种基本的数据抽象, 它对分层值的结构和操作方式施加了规律性。

一棵树有一个根 (root) 标签和一系列分支 (branches)。树的每个分支都是一棵树。没有树枝的树称为叶 (leaf)。树中包含的任何树都称为该树的子树 (sub-tree), 例如分支的分支。树的每个子树的根称为该树中的一个节点 (node)。

实现树抽象

```
>>> def tree(label, branches=[]):
    return [label] + list(branches)
```

一个树有一个标签值

和一个分支列表

```
>>> def label(tree):
    return tree[0]
```

```
>>> def branches(tree):
    return tree[1:]
```

```
def count_leaves(t):
    """Count the leaves of a tree."""
    if is_leaf(t):
        return 1
    else:
        branch_counts = [count_leaves(b) for b in branches(t)]
        return sum(branch_counts)
```

链表 (linked list):

链表是一个对象的链, 链里面每一个对象都持有一个值和指向下一个对象的链接。链表的最末对象的指向是空。

```
four = [1, [2, [3, [4, 'empty']]]
```

链表具有递归结构: 链表的其余部分是链表或 Empty。

我们可以定义一个抽象数据表示来验证、构造和选择链表的组件。

```
>>> empty = 'empty'
```

```

>>> def is_link(s):
    """s is a linked list if it is empty or a (first, rest) pair."""
    return s == empty or (len(s) == 2 and is_link(s[1]))
>>> def link(first, rest):
    """Construct a linked list from its first element and the rest."""
    assert is_link(rest), "rest must be a linked list."
    return [first, rest]
>>> def first(s):
    """Return the first element of a linked list s."""
    assert is_link(s), "first only applies to linked lists."
    assert s != empty, "empty linked list has no first element."
    return s[0]
>>> def rest(s):
    """Return the rest of the elements of a linked list s."""
    assert is_link(s), "rest only applies to linked lists."
    assert s != empty, "empty linked list has no rest."
    return s[1]

```

上述代码中，link 是构造子，first 和 rest 是选择子

满足序列抽象：长度和元素选择

```

>>> def len_link(s):
    """Return the length of linked list s."""
    length = 0
    while s != empty:
        s, length = rest(s), length + 1
    return length
>>> def getitem_link(s, i):
    """Return the element at index i of linked list s."""
    while i > 0:
        s, i = rest(s), i - 1
    return first(s)

```

利用递归可以很容易做到如链表的结合：

```

>>> def extend_link(s, t):
    """Return a list with the elements of s followed by those of t."""
    assert is_link(s) and is_link(t)
    if s == empty:
        return t
    else:
        return link(first(s), extend_link(rest(s), t))

>>> def apply_to_all_link(f, s):
    """Apply f to each element of s."""
    assert is_link(s)

```

```

if s == empty:
    return s
else:
    return link(f(first(s)), apply_to_all_link(f, rest(s)))
>>> apply_to_all_link(lambda x: x*x, four)
[1, [4, [9, [16, 'empty']]]]

>>> def keep_if_link(f, s):
    """Return a list with elements of s for which f(e) is true."""
    assert is_link(s)
    if s == empty:
        return s
    else:
        kept = keep_if_link(f, rest(s))
        if f(first(s)):
            return link(first(s), kept)
        else:
            return kept
>>> keep_if_link(lambda x: x%2 == 0, four)
[2, [4, 'empty']]

```

元组 (tuple):

元组也是一种 python 内建的序列，其一旦创建，后续无法改变其值。
以逗号分隔元素表达式的元组文本创建的，圆括号是可选的（但在实践中经常使用）。

和列表一样，元组内的元素可以为任意类型的对象。

```
>>> () # 0 elements
```

```
()
```

```
>>> (10,) # 1 element
```

```
(10,)
```

单个元素元组（注意逗号）

和列表一样，我们可以查询元组中元素的数量、索引其中的元素等
但不能更改其中的元素

但其某个元素如果是可变的，那么该值可以变化

4) 分支与循环 if 语句

5) 继承与多态

6) 类（面向对象）

2. 环境图

– 理解程序是怎么创建帧的

环境图：一个可以追踪（Track）一个程序运行时的绑定和状态变化的可视化工具。

帧（Frames）：

帧追踪着名字和值的绑定。

每一个函数调用都会有一个对应的帧。

全局帧（Global Frame）：就是最开始的帧，它不对应函数调用。

父帧（Parent Frame）：

某个函数的父帧就是定义这个函数语句的所在的帧。

如果你找不到一个变量名，你需要找到其父帧，如果还找不到，往其父帧的父帧查找，一直到全局帧。此时再找不到就是一个 Name Error。

帧的创建和求值顺序：

回顾求值规则：先对运算符求值、然后操作数，最后执行应用。

3.递归

– 怎么利用递归解决问题？

递归用来解决具有自我重复结构的问题。

关键就是找到这样的重复结构，将问题分解为更小的问题，并且定义出最小的问题。

递归是通过函数自身调用来解决问题的一种方法。可以将大问题拆解成小问题，直到解决小问题的方法已知，然后再将所得到的解用来解决大问题。

递归函数（Recursive Functions）：

一个函数被称为递归函数如果在自己的函数体内直接或者间接调用自己。这意味着执行一个递归函数的函数体往往会作用该函数多次。

递归其本质就是一种函数抽象。

递归树

– 递归的基本 case

递归的基本 case 就是当问题被拆解到足够小的规模时，可以直接解决，不再需要进一步递归拆解。

– 互递归是什么？

互递归是指多个函数相互调用的一种递归方式。在互递归中，相互调用的函数可能是同一函数的不同实例，也可能是互相调用的不同函数，它们相互协作完成任务。互递归常出现在处理复杂的数据结构时，可以将问题分解为更小且相互依存的子问题进行解决。

互递归（Mutual Recursion）：

如果两个函数 A 和 B，A 调用了（间接或直接）B，B 调用了（间接或直接）A，那么就称 A 和 B 是互递归的（Mutual Recursive）。与直接递归（direct recursion）直接调用自身不同，互递归是通过调用其他函数来间接的递归调用自己，因此也称为间接递归（indirect recursion）。

可以将互递归转化为直接递归：互递归可以通过将一个函数的代码内联进另一个函数中从而变为单个的递归函数。

4.生成器

– 生成器的作用和写法

生成器是一种特殊的函数，可以用来创建迭代器，它不会一次性生成所有结果，而是每次只返回一个结果，直到所有结果都被生成。这样可以节省大量内存空间。

生成器可以使用 `yield` 语句产生迭代器的一段代码块。生成器执行到 `yield` 语句时，会暂停执行，将结果返回给调用者，并保存当前的状态，下次调用时从上次暂停的状态继续执行。

生成器的写法：

```
```python
def generator_name(arguments):
 # some code
 yield result
 # some code
```
```

其中 `'yield'` 语句是生成器的核心，用于返回结果并保存当前状态。在调用生成器函数时不会执行函数体，而是返回一个生成器对象，可以通过 `'next()'` 函数或 `'for'` 循环迭代生成器，逐步获取生成器的结果。

例如，以下代码实现一个简单的生成器，可以以步长 2 生成从 1 开始的奇数：

```
```python
def odd_number():
 i = 1
 while True:
 yield i
 i += 2
```
```

调用生成器，迭代生成的奇数

```
odds = odd_number()
print(next(odds)) # 1
print(next(odds)) # 3
print(next(odds)) # 5
```
```

输出：

```
```
1
3
5
```
```

当生成器执行到 `yield` 语句时，将结果 1 返回给调用者，并保存当前状态，下次调用时从上次暂停的位置继续执行。因此，每次调用生成器函数都只会返回一个结果，直到生成器生成完所有结果。

生成器工作流程:

当生成器函数被调用时，其直接返回一个迭代器（没有进入函数体）。

当 `next()` 被调用时，其进入函数体，从上一轮的终止点（初始就是函数体的第一句），运行到下一轮的 `yield` 语句。如果找到了 `yield` 语句，那么其在下一句停止（这一次的终止点，作为下一轮的起点），并返回 `yield` 语句中的表达式的值。如果找不到 `yield` 语句，则在函数的末端终止，并产生一个 `StopIteration` 的异常。

### 递归的 yield

```
def factorial(n, accum):
 if n == 0:
 yield accum
 else:
 for result in factorial(n - 1, n * accum):
 yield result

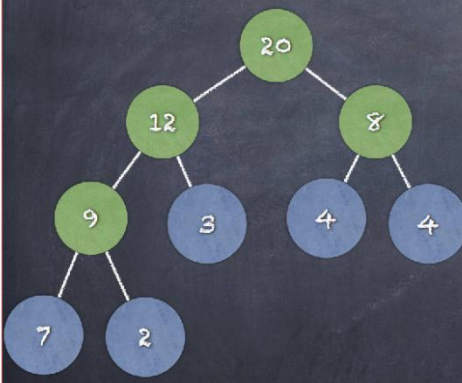
for num in factorial(3, 1):
 print(num)
```

yield from 还可以从 generator function 中获得 generator:

```
def factorial(n, accum):
 if n == 0:
 yield accum
 else:
 yield from factorial(n - 1, n * accum)

for num in factorial(3, 1):
 print(num)
```

### 树的递归生成器



```
A pre-order traversal of the tree:
def nodes(t):
 yield label(t)
 for c in branches(t):
 yield from nodes(c)

t = tree(20, [tree(12, [tree(9, [tree(7), tree(2)]), tree(3)]), tree(8, [tree(4), tree(4)])])

node_gen = nodes(t)
next(node_gen)
```

## 5. 数据抽象

– 数据抽象的概念

函数的抽象:

将计算过程抽象出来

一次定义，多次使用

高阶函数



## 递归

数据抽象：将复合数据对象的表示和处理分离，使得我们在使用这些对象时，不必对数据的具体形式做过多的假设。

图灵机，lambda 演算，字符串，树

关键思想：

1. “模块化”（structure）程序使得其操作在“抽象”的数据上，而不是某种特定类型的数据。

2. 具体的数据表示应该作为程序的独立部分（独立于对其使用的过程）。

有理数的例子，numerator，denominator

构造子，选择子

抽象界限（Abstraction Barriers）

抽象有层次之分，每一层次的抽象，应该明晰具体能够做的操作。层次分离了使用数据抽象的函数和实现数据抽象的函数。

## 6.面向对象部分

– 重载与重写

面向对象编程（OOP）提供了一种有效的数据抽象方式，其统一抽象出了信息和相应的行为三大特征：封装（encapsulation）、继承（Inheritance）、多态（polymorphism）。

类（class）：定义一个新的类型的模版

对象（object）：一个类的实例（instance）

实例变量（instance variables）：每个对象拥有的数据属性（attributes），以表达该对象的状态（也叫数据成员，members）

方法（method）：每个对象拥有的函数属性

类实例化 (Class instantiation)

- ① 类的实例化就是对象的构造 (Object construction)
- ② 当 `<class name> (args)` 被调用时，
  - ③ 1. 类 `<class name>` 的实例被创建
  - ③ 2. 类中的 `__init__` 方法被调用，新创建的实例被作为实参传入（名字为 `self`），同时，额外的参数 (`args`) 也被传入

构造子 (constructor)

## 方法调用 (invoking method)

- ❶ 可以获取对象实例变量的值的方法就是该对象的选择子 (selector) 或者叫访问子 (accessor)
- ❷ 可以改变对象实例变量的值的方法就是该对象的变异子 (mutator)

继承 (Inheritance) :

继承是一个关联多种类的技术。一个常见场景：两个类很相似，只有部分专有行为不同。

把两个类的所有内容都写一遍？更好的做法：一个特定的类可以将这些共同点作为一个通用类，额外附加其特定专用的行为即可。

继承 (Inheritance)

class <Name>(<Base Class>):

<suite>

创建一个类，名字为<Name>， 其是类<Base Class>的子类 (subclass)，类<Base Class> 也被称为该类的父类、基类或超类 (Superclass)。新创建的类继承了 (即含有) <Base Class>的属性。子类可以在<suite>添加自己的专有属性。子类也可以修改父类的属性 即重写 (overriding)。

重写变量：子类可以重新定义已在父类中的变量。

重写方法：子类可以重新定义已在父类中的方法。

使用基类中的方法：可以使用 `super()`方法来调用。

重写 `__init__`：同样，如果想要调用基类的 `__init__`方法，需要显式地调用 `super().__init__()`

多重继承 (Multiple inheritance)

继承的缺点：

打破了封装性：继承强制对子类开发者知道其父类的内部信息

继承所带来的代价：需要存储超类的变量、构造子、方法、但可能只有少量的超类方法被用到

重载(Overload)：是一个类中多态性的一种表现，指同一个类中不同的函数使用相同的函数名，但是函数的参数个数或类型不同。可以有不同的返回类型;可以

有不同的访问修饰符;可以抛出不同的异常。调用的时候根据函数的参数来区别不同的函数。

**重写 (Override):** 是父类与子类之间的多态性, 是子类对父类函数的重新实现。函数名和参数与父类一样, 子类与父类函数体内容不一样。子类返回的类型必须与父类保持一致; 子类方法访问修饰符的限制一定要大于父类方法的访问修饰, 子类重写方法一定不能抛出新的检查异常或者比被父类方法申明更加宽泛的检查型异常。

**区别:** 重载实现于一个类中;重写实现于子类中。

#### 四. Java

##### 面向对象

- Java 的面向对象和 Python 有什么不同?
- 什么是绑定? 有哪些分类?

Java 和 Python 都是面向对象编程语言, 但在面向对象的实现和语法上存在一些不同。

首先, 在 Java 中, 所有的代码都必须放在类中, 而 Python 中既可以放在类中, 也可以独立于类之外。此外, Java 是一门静态类型语言, 变量和对象在编译时就确定其数据类型, 而 Python 是一门动态类型语言, 变量和对象在运行时动态确定其数据类型。Java 通过引用传递对象, 而 Python 中传递的是对象的引用。

另外, 在 Java 中, 所有的方法和变量必须包含在类中, 可以通过类名来调用方法和变量。而 Python 中, 方法和变量可以直接在实例对象上调用, 没有必要使用类名。

Java 中的绑定是指在编译时或运行时将代码与特定实体 (如变量、方法、类) 相关联的过程。Java 中有三种类型的绑定:

1. **静态绑定 (Static Binding):** 也称为早期绑定或编译时绑定, 是指在编译时确定方法或变量的使用, 不需要等到运行时再确定。
2. **动态绑定 (Dynamic Binding):** 也称为晚期绑定或运行时绑定, 是指在运行时通过对象的类型来确定方法或变量的使用。

其中, 动态绑定在继承和多态等方面比较常见, 可以让程序在运行时根据实际的对象类型调用相应的方法。而静态绑定通常用于提高程序运行效率。

例如, 以下代码演示了 Java 的静态绑定和动态绑定:

```
```java
class Animal {
```

```

    void makeSound() {
        System.out.println("Animal makes sound.");
    }
}

class Dog extends Animal {
    void makeSound() {
        System.out.println("Dog barks.");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal an = new Animal();
        Dog dog = new Dog();
        Animal anRef = dog;

        an.makeSound();    // Animal makes sound. (静态绑定)
        dog.makeSound();    // Dog barks. (静态绑定)
        anRef.makeSound();  // Dog barks. (动态绑定)
    }
}

```

在上述代码中，Animal 类和 Dog 类之间存在继承关系，重写了 Animal 类的 `makeSound()` 方法。在主函数中，创建了 Animal 对象和 Dog 对象，并将 Dog 对象的引用赋值给 Animal 类型的变量。通过调用三个对象的 `makeSound()` 方法，可以看到在静态绑定时直接根据对象的类型确定调用哪个方法，在动态绑定时根据对象的实际类型来确定调用哪个方法。

拷贝

浅拷贝（shallow copy）

创建一个新的对象，然后把原对象中的元素的引用拷贝进去

深拷贝（deep copy）

创建一个新的对象，然后递归地把原对象中的所有元素拷贝进去

可变性和不可变性

同一性和相等性

Scope 作用域：全局：

作用域规则

动作	全局帧的代码	本地帧的代码
可以访问全局帧里绑定的名字?	可以	可以
对全局帧绑定的名字进行重新赋值?	可以	不可以 (除非申明 <code>global</code>)

全局变量重新赋值

```
current = 0

def count():
    global current
    current = current + 1
    print("Count:", current)

count()
count()
```

尽量避免使用 `global`

嵌套的作用域:

作用域规则

访问绑定在 <code>enclosing function</code> 里的名字?	可以
重新赋值绑定在 <code>enclosing function</code> 的名字?	不可以 (除非申明 <code>nonlocal</code>)

尽量避免使用 `nonlocal`